

Autonomous Content Engine

Architecture and Implementation

Specification

Building an autonomous omnipresence system identical to makerzz.space requires orchestrating a decoupled, event-driven pipeline that converts a single profile URL into a self-sustaining, thirty-day multi-platform publishing cadence. The system does not function as an interactive conversational assistant; it operates as an asynchronous media compilation engine. By chaining distributed scraping, search-grounded pattern extraction, constrained generative reasoning, headless image synthesis, avatar generation, and deterministic video compositing, the engine programmatically renders and publishes multimedia assets across thirteen social networks without ongoing manual intervention.

The underlying architecture relies on five specialized infrastructure layers coordinated through asynchronous job queues: Apify for competitive data harvesting, Perplexity Sonar coupled with Anthropic Claude for viral pattern identification and prompt compilation, Fal.ai for programmatic carousel generation, the HeyGen API combined with the open-source HyperFrames framework for deterministic video compositing, and Ayrshare for multi-platform distribution. A central technical and business innovation of the platform is the bifurcation between metered server-side reasoning and unmetered social delivery, allowing the platform to absorb fixed-cost API posting overhead while charging premium credit margins on computationally intensive generative tasks. Operating this system requires isolating long-running browser processes and external API polling loops from the user interface. The end-to-end pipeline executes in ninety to one hundred thirty seconds, persisting intermediate artifacts to S3-compatible object storage while updating an interactive Next.js console via streaming protocols.

System Pipeline and Core Operational Flow

The platform transitions an initial social link into scheduled multi-format media across nine distinct operational stages. Each stage is isolated through message broker queues, ensuring idempotency, credit verification, and state persistence.

Stage	Operational Domain	Primary Technology / API	Processing Latency	Metered Credit Cost	Output Artifact
1	Profile & Competitor Ingestion	Apify Actors (Instagram, TikTok, YouTube)	30–60 seconds	40 credits (deep scan)	Normalized JSON corpus of up to 50 competitor profiles
2	Pattern Extraction & Hook Mining	Perplexity Sonar API + Claude 3.5 Sonnet	20–30 seconds	Included in strategy run	Scored viral hook grammars, engagement patterns,

Stage	Operational Domain	Primary Technology / API	Processing Latency	Metered Credit Cost	Output Artifact
					optimal posting windows
3	30-Day Strategy & Calendar Compilation	Claude 3.5 Sonnet (Structured Output)	15–25 seconds	122 credits (baseline strategy call)	30-day scheduled matrix (topics, frameworks, format tags)
4	Script & Reel Edit Plan Generation	Claude 3.5 Sonnet + GPT-4o Verification	10–15 seconds	12 credits (script) + 20 credits (edit plan)	Spoken text transcripts, visual shot-lists, sound effects map
5	Carousel Prompt Compilation & Rendering	Claude 3.5 Sonnet + Fal.ai (Flux/Recraft)	15–30 seconds	30 credits (prompts) + 24 credits (8 standard slides)	8-slide high-resolution carousel PNG package
6	Digital Twin Avatar Synthesis	HeyGen API v2 (/v2/video/generate)	60–120 seconds	202 credits (60s @ 1080p) / 215 credits (4K)	Transparent or solid-background MP4 avatar talking-head clip
7	Deterministic Video Compositing	HyperFrames Engine + FFmpeg on Lambda	30–60 seconds	0 credits (internal compute execution)	9:16 vertical MP4 video with burned-in kinetic typography and split layouts
8	Human-in-the-Loop Approval & Refill	Next.js Console + SSE / WebSocket Stream	User-dependent	0 credits	State transition: AWAITING_APPROVAL to SCHEDULED
9	Unmetered 13-Platform Publishing	Ayrshare API (/api/post REST endpoint)	2–5 seconds	0 credits (unmetered platform layer)	Live published URLs across 13 social networks

By maintaining deterministic job boundaries, failures in expensive visual stages (such as avatar rendering) do not invalidate upstream strategic data. Every run terminates in an exported PDF audit report documenting extracted patterns, compiled scripts, generated visual assets, and assigned publication slots.

Competitive Ingestion and Algorithmic Pattern Mining

The ingestion pipeline begins when a user inputs a profile URL representing their own account or up to fifty competitor accounts. The objective is not to replicate existing posts verbatim, but to deconstruct the high-performing structural mechanisms—a technique structured around extracting the operational pattern rather than the literal copy.

Scraping and Ingestion Normalization

The system queries specialized Apify actors (apify/instagram-profile-scraper, apify/tiktok-scraper, and apify/youtube-scraper) to extract historical performance metrics across the target accounts. To bypass anti-scraping protections and dynamic platform rate-limiting, requests are distributed across residential proxy pools. The scraper extracts the top fifty most engaging reels, shorts, and carousel posts based on engagement velocity:

$$\text{Engagement Rate} = \frac{\text{Likes} + \text{Comments}}{\text{Views}} \times 2 + \text{Shares} \times 4$$

Unstructured platform outputs are ingested and normalized into a standard JSON intermediate representation:

```
{
  "platform": "instagram",
  "creator_id": "alexhormozi",
  "post_id": "17878176260289172",
  "metrics": {
    "views": 1420000,
    "likes": 86400,
    "comments": 3120,
    "shares": 14200,
    "engagement_rate": 0.073
  },
  "content": {
    "caption": "Most business owners don't have a traffic problem...",
    "hook_text": "Most business owners don't have a traffic problem.",
    "duration_seconds": 48.2,
    "format": "split_screen_talking_head",
    "published_timestamp": 1709284800
  }
}
```

The data pipeline enforces strict data integrity. If platform privacy restrictions prevent the extraction of verified view counts or share figures, the system logs the fields as null and notes them as unretrievable in the database rather than interpolating synthetic figures.

Hook Grammar and Pattern Extraction

Once the normalized corpus is constructed, the engine executes a comparative pattern extraction pipeline combining the Perplexity Sonar API (sonar-pro) and Anthropic Claude 3.5 Sonnet. Sonar crawls live semantic search indices to establish the broader market context of the niche, identifying trending topic clusters and high-performing content formats. Claude 3.5 Sonnet evaluates the corpus against a specialized Hook Grammar specification, identifying syntactic patterns that halt scrolling within the first three seconds of playback:

$$\text{Hook Grammar} = [\text{Disruption Marker}] + [\text{Target Identity}] + [\text{Counter-Intuitive Truth}] + [\text{Stakes Anchor}]$$

The pattern extractor parses content into three structural units: the Hook (the initial 15 spoken words or on-screen graphic), the System (the mechanistic explanation, framework, or operational protocol), and the Call to Action (the conversion mechanism). These patterns are scored and categorized into eight ranked promotional angles, which are written to PostgreSQL to seed the downstream calendar compilation engine.

Automated Content Strategy and Verification-Gated Scripting

The content orchestration layer translates extracted patterns into a continuous thirty-day publishing schedule. To prevent the delivery of generic or ungrounded generative copy, the script engine couples Claude 3.5 Sonnet with a secondary GPT-4o verification gate.

Calendar Matrix Compilation

Claude 3.5 Sonnet maps the eight ranked angles across a thirty-day content matrix. The engine structures the schedule around the "Hormozi Rule" framework, prioritizing high-volume omnipresence, tactical execution steps, contrarian viewpoints, and direct business mechanics over high-level advice.

To prevent social network algorithms from penalizing synchronized automation, the scheduler calculates optimal publishing times using competitor data and introduces randomized jitter:
$$\text{Scheduled Time} = \text{Optimal Window Peak} \pm \text{UniformRandom}(3, 17) \text{ minutes}$$

The resulting matrix allocates alternating content formats (visual carousels, vertical video reels, and text packs) to each calendar slot.

Verification-Gated Script Engine

Each script and carousel blueprint undergoes programmatic generation and validation. Claude 3.5 Sonnet generates the spoken scripts, shot-lists, and platform captions, which are transmitted to a GPT-4o verifier agent configured with structured JSON schema boundaries:

```
{
  "$schema":
    "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "day_index": { "type": "integer", "minimum": 1, "maximum": 30 },
    "scheduled_timestamp": { "type": "string", "format": "date-time"
  },
  "format_type": { "type": "string", "enum": ["REEL",
    "CAROUSEL", "TEXT_PACK"] },
  "hook": {
    "spoken_script": { "type": "string", "maxLength": 120 },
    "on_screen_text": { "type": "string", "maxLength": 50 }
```

```

    },
    "body_beats": {
      "type": "array",
      "items": {
        "beat_index": { "type": "integer" },
        "spoken_text": { "type": "string" },
        "visual_direction": { "type": "string" },
        "target_duration_seconds": { "type": "number" }
      }
    },
    "cta": {
      "spoken_script": { "type": "string", "maxLength": 80 },
      "action_keyword": { "type": "string" }
    },
    "captions": {
      "type": "object",
      "properties": {
        "instagram": { "type": "string" },
        "linkedin": { "type": "string" },
        "x": { "type": "string", "maxLength": 280 },
        "youtube": { "type": "string" }
      },
      "required": ["instagram", "linkedin", "x", "youtube"]
    }
  },
  "required": ["day_index", "scheduled_timestamp", "format_type",
    "hook", "body_beats", "cta", "captions"]
}

```

The verifier evaluates the script against strict delivery constraints: spoken cadence must not exceed 2.5 words per second, hooks must deliver the primary premise within the first 6 words, and copy must not contain conversational AI filler phrases. If the verifier rejects the script, the worker initiates an automated regeneration loop, ensuring credit deduction occurs only after the asset passes validation.

Programmatic Carousel Synthesis via Diffusion and Canvas Engines

Carousels are generated as ready-to-publish image sets rather than descriptive outlines. The architecture implements two distinct generation pathways based on the requested visual style: Direct Diffusion Generation via Fal.ai and Deterministic Canvas Rendering via Puppeteer.

Direct Diffusion Rendering via Fal.ai

For photographic, editorial, or high-concept visual carousels, the Carousel Prompt Compiler expands the verified copy into eight coordinated image generation prompts. These prompts are dispatched in parallel to Fal.ai endpoints running Flux Dev, Flux Schnell, or Recraft v3:

```
{
  "prompt": "Editorial brutalist presentation slide, bold typographic hierarchy, top tag reads '03/06 BUILD THE SYSTEM', center isometric schematic of data blocks connecting, muted beige background with faint grid lines, studio lighting, hyper-minimal aesthetic --ar 4:5 --raw",
  "image_size": { "width": 1080, "height": 1350 },
  "num_inference_steps": 28,
  "guidance_scale": 3.5,
  "seed": 421098
}
```

Recraft v3 is deployed specifically when accurate typographic rendering must be baked directly into rasterized outputs. Standard slides consume 3 credits each (24 credits for an 8-slide set), while high-quality and 4K upscaling triggers 2times and 4times credit multipliers respectively.

Headless Canvas Template Rendering

When typography, layout geometry, and brand identity require absolute precision, the platform routes generation through an internal Puppeteer rendering pool. An HTML and CSS compiler injects brand fonts, color variables, and SVG icons into structured slide templates. Puppeteer manages a cluster of headless Chromium processes, rendering 1080x1350 pixel viewports at a device scale factor of 2 to yield razor-sharp 4:5 graphics:

```
import puppeteer from "puppeteer";

export async function renderSlideDeck(slidesHtml: string[]):
Promise<Buffer[]> {
  const browser = await puppeteer.launch({
    headless: true,
    args: ["--no-sandbox", "--disable-setuid-sandbox",
"--disable-dev-shm-usage"]
  });
  const page = await browser.newPage();
  await page.setViewport({ width: 1080, height: 1350,
deviceScaleFactor: 2 });

  const buffers: Buffer[] = [];
  for (const html of slidesHtml) {
    await page.setContent(html, { waitUntil: "networkidle0" });
    const buffer = await page.screenshot({ type: "png",
omitBackground: false });
    buffers.push(buffer as Buffer);
  }
  await browser.close();
  return buffers;
}
```

This dual-engine architecture allows operators to balance stylistic expression against precise

layout governance based on specific campaign requirements.

Synthetic Avatar Production and Deterministic Video Compositing

Vertical video creation automates talking-head filming and post-production editing into a single pipeline. The architecture decouples avatar generation from timeline composition to maintain determinism and optimize compute efficiency.

Video Assembly Workflow

The assembly process proceeds sequentially from the verified script to the published MP4 file. The table below outlines the input triggers, processing mechanisms, and resulting media states across the automated editing pipeline.

Pipeline Stage	Upstream Input	Execution Mechanism	Output Asset
Avatar Generation	Spoken text script and selected voice ID	HeyGen API v2 request with solid green-screen background (#00FF00)	Raw 1:1 or 9:16 MP4 avatar video clip
Motion Layout	Verified shot-list and visual context tags	Claude 3.5 Sonnet compiles a HyperFrames HTML/CSS/GSAP composition document	Standalone HTML5 video composition bundle (index.html)
Deterministic Rendering	HTML bundle and downloaded media assets	@hyperframes/producer steps the Chromium frame clock at 30 fps via beginFrame	Sequenced raw frame captures and extracted audio stream
Final Encoding	Frame sequence and audio assets	FFmpeg composites green-screen chroma key, mixes audio tracks, and encodes H.264	Production-ready 1080x1920 MP4 vertical video

Avatar Video Synthesis via HeyGen API

The verified spoken script is sent to the HeyGen API v2 endpoint (POST <https://api.heygen.com/v2/video/generate>). The payload configures the designated digital avatar (either a stock presenter or a custom trained clone), voice ID, speech rate, and a chroma-key green background to allow downstream compositing:

```
{
  "video_inputs": [
    {
      "character": {
        "type": "avatar",
        "avatar_id": "Alex_Hormozi_Clone_v4",
        "avatar_style": "normal"
      },
      "voice": {
        "type": "text",
        "input_text": "Stop asking AI to design your entire business from scratch. Do this instead.",
        "voice_id": "en-US-AlexNatural",
        "speed": 1.05
      },
      "background": {
```

```

        "type": "color",
        "value": "#00FF00"
      }
    ],
    "dimension": {
      "width": 1080,
      "height": 1080
    },
    "aspect_ratio": "1:1",
    "test": false
  }
}

```

The system captures the returned `video_id` and registers an asynchronous callback handler via HeyGen webhooks (`avatar_video.success`). Video synthesis is the most compute-intensive external call in the pipeline, consuming 202 credits for a 60-second clip at 1080p and 215 credits at 4K.

Deterministic Compositing via HyperFrames

Rather than using traditional video editing software or generic FFmpeg filters, post-production is driven by HyperFrames—an open-source framework developed by HeyGen (Apache 2.0 license) that turns HTML, CSS, and GSAP animations into deterministic MP4 videos.

HyperFrames guarantees that identical composition code produces identical video output across rendering environments. The engine eliminates real-time clock references such as `Date.now()`,

`requ[span_43](start_span)[span_43](end_span)[span_55](start_span)[span_55](end_span)estA` `animationFrame`, or unseeded `Math.random()`, advancing time exclusively via integer-based frame mathematics:

$$\text{time} = \frac{\lfloor \text{frame} \rfloor}{\text{fps}}$$

During rendering, a headless Chromium instance seeks the root GSAP timeline to the exact calculated timestamp for each frame, captures the rendered DOM using `HeadlessExperimental.beginFrame`, and streams the raw buffer to FFmpeg.

The standard makerzz editing aesthetic uses a split-screen vertical layout (1080x1920): the top half presents dynamic visuals or B-roll, the bottom half displays the talking-head avatar, and burned-in kinetic typography overlays the boundary. The entire scene is defined within a single HTML file:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.2/gsap.min.js"><
/script>
  <style>
    :root {
      --bg: #0C0D0E;
      --accent: #FF4A22;

```

```

    --text: #F4F4F5;
    --font-display: 'Inter', sans-serif;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
body { background: var(--bg); overflow: hidden; }
#stage {
    position: relative;
    width: 1080px;
    height: 1920px;
    overflow: hidden;
    background: var(--bg);
    font-family: var(--font-display);
}
.top-panel {
    position: absolute;
    top: 0;
    left: 0;
    width: 1080px;
    height: 960px;
    overflow: hidden;
}
.top-panel video, .top-panel img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
.bottom-panel {
    position: absolute;
    top: 960px;
    left: 0;
    width: 1080px;
    height: 960px;
    overflow: hidden;
    background: #141517;
    display: flex;
    justify-content: center;
    align-items: flex-end;
}
.avatar-video {
    height: 100%;
    object-fit: cover;
}
.kinetic-caption {
    position: absolute;
    top: 910px;
    width: 100%;
    text-align: center;
    font-size: 56px;
}

```

```

        font-weight: 900;
        text-transform: uppercase;
        color: #FFFFFF;
        text-shadow: 0 4px 20px rgba(0,0,0,0.8);
        z-index: 100;
    }
    .highlight { color: var(--accent); }
</style>
</head>
<body>
    <div id="stage"
        data-composition-id="makerzz-split-edit"
        data-start="0"
        data-duration="40"
        data-width="1080"
        data-height="1920">

        <div class="top-panel clip" data-start="0" data-duration="40">
            
        </div>

        <div class="bottom-panel clip" data-start="0" data-duration="40">
            <video class="avatar-video" src="./assets/heygen_avatar.mp4"
muted></video>
        </div>

        <div id="caption-box" class="kinetic-caption clip" data-start="0"
data-duration="40">
            <span id="caption-text">STOP ASKING AI</span>
        </div>
    </div>

    <script>
        const rootTimeline = gsap.timeline({ paused: true });
        window.__timelines = [rootTimeline];

        rootTimeline.to("#caption-text", {
            duration: 0.5,
            scale: 1.15,
            color: "#FF4A22",
            repeat: -1,
            yoyo: true,
            ease: "power2.out"
        }, 0);

        rootTimeline.to("#broll-media", {
            duration: 40,

```

```

        scale: 1.25,
        ease: "none"
      }, 0);
    </script>
  </body>
</html>

```

The composition worker executes `@hyperframes/producer` within an AWS Lambda container or Google Cloud Run task, rendering the final vertical video in under forty seconds:

```

import { createRenderJob, executeRenderJob } from
"@hyperframes/producer";

export async function executeVideoCompositing(projectDirectory:
string, outputMp4Path: string): Promise<void> {
  const job = createRenderJob({
    fps: 30,
    quality: "high",
    format: "mp4"
  });

  await executeRenderJob(
    job,
    projectDirectory,
    outputMp4Path,
    (currentJob, message) => {
      console.log(`Render progress:
${Math.round(currentJob.progress)}% - ${message}`);
    }
  );
}

```

Unmetered Multi-Platform Distribution and Continuous Queue Refill

The distribution layer delivers rendered assets across external social networks on an automated publishing schedule.

Multi-Platform Dispatch via Ayrshare

The platform supports thirteen distribution channels: Instagram, YouTube, LinkedIn, TikTok, Facebook, Threads, X, Pinterest, Bluesky, Reddit, Telegram, Discord, and Google Business Profile.

Rather than managing separate OAuth tokens, refresh lifecycles, and changing payload formats across thirteen independent APIs, the system routes publishing calls through Ayrshare's unified social gateway. Ayrshare validates video container constraints, resizes imagery to meet platform specifications, manages rate limits, and automates platform-specific actions such as inserting

first comments on TikTok and Instagram.

```
import
axios[span_97](start_span)[span_97](end_span)[span_100](start_span)[span_100](end_span) from
"a[span_31](start_span)[span_31](end_span)xios";

interface AyrsharePostRequest {
  post: string;
  platforms: string[];
  mediaUrls: string[];
  scheduleDate?: string;
  requiresApproval?: boolean;
  youtubeVisibility?: "public" | "unlisted" | "private";
  youtubeOptions?: { title: string };
  autoHashtag?: { max: number; position: "auto" | "end" };
}

export async function dispatchOmnipresencePost(payload:
AyrsharePostRequest, userApiKey: string) {
  const response = await
  axios.post("https://api.ayrshare.com/api/post", payload, {
    headers: {
      "Authorization": `Bearer ${userApiKey}`,
      "Content-Type": "application/json"
    }
  });
  return response.data;
}
```

The Unmetered Economics Model

A key design principle of the makerzz platform is that publishing carries zero credit cost. While AI reasoning (Claude, GPT) and visual rendering (Fal.ai, HeyGen) scale on variable usage-based billing, enterprise social aggregation gateways bill platform operators a flat monthly fee per connected profile.

Because Ayrshare bills a static rate per active profile, marginal publishing costs do not scale with posting volume. Metering outbound social posts would artificially suppress platform engagement, so publishing costs are bundled directly into the base subscription tier.

Queue Management and Continuous Refill Automation

To maintain consistent account visibility, the platform ensures the scheduled queue never runs empty. A cron daemon audits active accounts daily. When an account's forward posting queue falls below seven days of scheduled content, the supervisor automatically initiates the subsequent thirty-day strategy cycle.

The system supports two operational modes:

- Autonomous Mode: Scraping, generation, rendering, and scheduling run without

- intervention, maintaining an active posting horizon indefinitely.
- Human-in-the-Loop Mode: The pipeline generates all scripts, slide decks, and video edits, pausing in an `AWAITING_APPROVAL` state in the web dashboard. The user reviews the queue and commits the schedule with a single confirmation.

Every completed generation cycle produces an auditable PDF report documenting inputs, model outputs, asset links, and scheduled publishing slots.

Relational Data Modeling and Distributed Task Orchestration

The system coordinates scraping tasks, large language model completions, diffusion image generation, and video compositing across an event-driven database and job queue architecture.

Relational Schema Design

The relational database layer is implemented in PostgreSQL, managing users, credit wallets, connected profiles, generation batches, and published items:

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE credit_wallets (
    user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    balance INTEGER NOT NULL DEFAULT 0,
    tier VARCHAR(50) NOT NULL DEFAULT
'BASIC', [span_34] (start_span) [span_34] (end_span)
    resets_at TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT
C[span_64] (start_span) [span_64] (end_span) [span_67] (start_span) [span_67
] (end_span)URRENT_TIMESTAMP
);

CREATE TABLE credit_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    amount INTEGER NOT NULL,
    reason VARCHAR(100) NOT NULL,
    reference_id UUID,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE connected_profiles (
```

```

        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        user_id UUID REFERENCES users(id) ON DELETE CASCADE,
        platform VARCHAR(50) NOT NULL,
        external_profile_id VARCHAR(255) NOT NULL,
        profile_handle VARCHAR(255) NOT NULL,
        ayrshare_profile_key VARCHAR(255) NOT NULL,
        is_competitor BOOLEAN DEFAULT FALSE,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE strategy_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    target_profile_id UUID REFERENCES connected_profiles(id),
    status VARCHAR(50) NOT NULL,
    audit_pdf_url TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE content_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    strategy_run_id UUID REFERENCES strategy_runs(id) ON DELETE
CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    format_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    scheduled_at TIMESTAMP WITH TIME ZONE NOT NULL,
    script_payload JSONB NOT NULL,
    edit_plan JSONB,
    rendered_assets JSONB,
    ayrshare_post_id VARCHAR(255),
    error_message TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

Distributed Queue Architecture via BullMQ

Background tasks are distributed across five dedicated Redis BullMQ queues: ScrapeQueue, StrategyQueue, ImageRenderQueue, HeyGenAvatarQueue, and HyperFrameCompositeQueue. The compositing worker runs on GPU-enabled or high-compute serverless container instances, downloading source assets from S3, running the HyperFrames headless engine, uploading finished MP4s, and updating database state:

```

import { Worker, Job } from "bullmq";
import { executeVideoCompositing } from "../hyperframe-service";
import { db } from "../database";

export const videoCompositeWorker = new Worker(

```

```

"HyperFrameCompositeQueue",
async (job: Job) => {
  const { contentItemId, projectDir, outputS3Key } = job.data;

  await db.updateContentStatus(contentItemId, "COMPOSITING");

  try {
    const localOutputFile = `/tmp/renderers/${contentItemId}.mp4`;

[span_65](start_span)[span_65](end_span)[span_68](start_span)[span_68]
(end_span)    await executeVideoCompositing(projectDir,
localOutputFile);

    const s3Url = await uploadToStorage(localOutputFile,
outputS3Key);
    await db.saveRenderedAsset(contentItemId, { videoUrl: s3Url });
    await db.updateContentStatus(contentItemId,
"AWAITING_APPROVAL");
  } catch (err: any) {
    await db.logFailure(contentItemId, err.message);
    throw err;
  }
},
{ concurrency: 4, connection: { host: "localhost", port: 6379 } }
);

```

Credit balances are checked and debited atomically prior to job creation. If an account balance reaches zero, the scheduler halts processing immediately to prevent unexpected third-party API charges.

Unit Economics, Metered Computation, and Margin Architecture

The platform sustains gross margins above 80% by separating computational reasoning from visual asset rendering in its pricing model. Pricing tiers and credit allocations are designed to absorb API expenses across varying usage patterns.

System Operation	User Metered Credit Cost	Internal Token / Compute Usage	Upstream Provider Direct Cost	Estimated Revenue Per Call (At \$0.053/credit)	Platform Gross Margin
Complete 30-Day Strategy Run	122 credits	80k Claude Tokens + Perplexity Search	~\$0.35 total (Claude + Sonar)	\$6.47	94.6%
Competitor Scraping (50	40 credits	Apify Actor Execution (150	~\$0.10	\$2.12	95.3%

System Operation	User Metered Credit Cost	Internal Token / Compute Usage	Upstream Provider Direct Cost	Estimated Revenue Per Call (At \$0.053/credit)	Platform Gross Margin
profiles)		requests)			
Carousel Prompt Compilation	30 credits	12k Claude 3.5 Sonnet Tokens	~\$0.04	\$1.59	97.5%
Spoken Script + Edit Plan	32 credits	15k Claude Tokens + 5k GPT-4o	~\$0.06	\$1.70	96.5%
8-Slide Carousel Render (Standard)	24 credits (3 cr/slide)	8 Fal.ai Generations (Flux Schnell)	~\$0.024 (\$0.003/image)	\$1.27	98.1%
8-Slide Carousel Render (4K)	96 credits (12 cr/slide)	8 Fal.ai Generations + 4K Upscale	~\$0.12 total	\$5.09	97.6%
60-Second Avatar Video (1080p)	202 credits	1 minute HeyGen API v2 Video Render	~\$2.00 (HeyGen API enterprise tier)	\$10.70	81.3%
60-Second Video (4K)	215 credits	1 minute HeyGen API 4K Render	~\$2.50	\$11.40	78.1%
HyperFrames Video Composition	0 credits	AWS Lambda Container (45s execution)	~\$0.003 (Compute/Memory)	\$0.00 (Included)	N/A
Social Distribution (13 platforms)	0 credits	Ayrshare API Post Dispatch	~\$0.00 (Amortized per-profile base)	\$0.00 (Included)	N/A

Subscription Tiers and Unit Economics

The platform provides three monthly subscription tiers: Basic (\$24/month for 400 credits), Visionary (\$79/month for 1,500 credits at ~\$0.053/credit), and Agentic (\$199/month for 5,000 credits at ~\$0.040/credit).

A standard user on the Visionary tier (\$79/month) typically runs 1 full niche audit (40 credits), 4 weekly trend refreshes (60 credits), 20 script generations (240 credits), 12 reel edit plans (240 credits), 10 carousel prompt packs (300 credits), and renders 10 standard 8-slide carousels (240 credits), consuming roughly 1,120 credits. The total upstream infrastructure cost for this usage volume across Apify, Claude, Perplexity, Fal.ai, and amortized Ayrshare fees comes to approximately \$6.50, yielding an operating gross margin above 90%.

Even when users produce avatar videos, the 202-credit pricing per 60-second video ensures a gross margin above 80% on each clip. The unmetered publishing model eliminates friction for high-frequency posters while maintaining stable profit margins through upfront credit debits. The architecture of makerzz.space demonstrates that building an automated social media engine requires treating content generation as a deterministic software compilation problem

rather than an interactive text-generation workflow. By integrating web-scale data extraction via Apify, semantic hook classification via Perplexity and Claude, deterministic asset rendering through Fal.ai and HyperFrames, and multi-network publishing via Ayrshare, the platform automates end-to-end social media operations into an autonomous, self-sustaining pipeline. Long-term operational reliability depends on maintaining isolated job queues, strict schema validation across generation stages, and an economic model that aligns compute usage directly with platform profitability.

Works cited

1. makerzz — Grow on social media on autopilot — makerzz, <https://makerzz.space/> 2. Piyush.glitch - Facebook, <https://www.facebook.com/1129089536955929/> 3. heygen-com/hyperframes: Write HTML. Render video. Built for agents., <https://github.com/heygen-com/hyperframes> 4. Unlock Heygen AI via API: Create Stunning Content in Minute - Apidog, <https://apidog.com/blog/heygen-api/> 5. Post API Overview - Ayrshare API Documentation, <https://www.ayrshare.com/docs/apis/post/overview> 6. Plans - makerzz, <https://makerzz.space/plans> 7. hyperframes/docs/packages/producer.mdx at main - GitHub, <https://github.com/heygen-com/hyperframes/blob/main/docs/packages/producer.mdx> 8. How to Build an AI Video Production Workflow with Claude Code, <https://www.mindstudio.ai/blog/ai-video-production-workflow-claude-code-heygen> 9. Deterministic Rendering - HyperFrames - HeyGen, <https://hyperframes.heygen.com/concepts/determinism> 10. Instagram Profile Scraper - Apify, <https://apify.com/apify/instagram-profile-scraper> 11. Instagram Post Scraper - Apify, <https://apify.com/apify/instagram-post-scraper> 12. Sonar API - Perplexity, <https://docs.perplexity.ai/docs/sonar/quickstart> 13. Introducing the Sonar Pro API by Perplexity, <https://www.perplexity.ai/hub/blog/introducing-the-sonar-pro-api> 14. hyperframes/docs/guides/open-design-hyperframes.md at main, <https://github.com/heygen-com/hyperframes/blob/main/docs/guides/open-design-hyperframes.md> 15. Skills - Bingran You, <https://bingran.ai/skills> 16. hyperframes-launch-video/STORYBOARD.md at main - GitHub, <https://github.com/heygen-com/hyperframes-launch-video/blob/main/STORYBOARD.md> 17. heygen-com/hyperframes-launches - GitHub, <https://github.com/heygen-com/hyperframes-launches> 18. Claude + HeyGen: What the Integration Can (and Can't) Do in 2026, <https://www.usecarly.com/blog/claude-heygen-integration/> 19. CLAUDE.md - heygen-com/hyperframes · GitHub, <https://github.com/heygen-com/hyperframes/blob/main/CLAUDE.md> 20. Publish a Post API: Post to Any Social Network | Ayrshare Docs, <https://www.ayrshare.com/docs/apis/post/post> 21. Quick Start Guide - Ayrshare API Documentation, <https://www.ayrshare.com/docs/quickstart> 22. ayrshare/ayrshare_flutter - GitHub, https://github.com/ayrshare/ayrshare_flutter